

Programmieren für Kids

JavaScript für Kids

Modul 3

Zufall und Entscheidungen



Trainer: Wolfgang Schedl

Kurz zurückgeblickt

Deine Knöpfe können schon etwas auslösen. Aber sie machen jedes Mal **genau dasselbe**. Heute wird der Computer schlau. Er lernt zwei neue Superkräfte:

- **Würfeln:** jedes Mal etwas anderes
- **Entscheiden:** wenn das, dann tu jenes



Das baust du in diesem Modul

Zwei echte Spiele gegen den Computer:

- eine **Wahrsager-Kugel**, die zufällige Antworten gibt
- **Schere, Stein, Papier** gegen den Computer

Zum ersten Mal weißt du vorher nicht, was passiert.



Kapitel 1

Listen: viele Dinge auf einmal

¹ ² ³ ≡ Was ist eine Liste?

Eine Variable hält *ein* Ding. Aber oft brauchst du **viele** auf einmal, zum Beispiel alle Antworten deiner Wahrsager-Kugel. Dafür gibt es die **Liste**: eine Kiste, die viele Dinge der Reihe nach hält.

Eine Liste bauen

```
const antworten = ["Ja", "Nein", "Vielleicht"]
```

- die eckigen Klammern [] machen eine Liste
- die Dinge stehen darin, mit Komma getrennt

Ein Ding herausholen

Du holst ein Ding über seine **Position**:

```
console.log(antworten[0])    // Ja  
console.log(antworten[1])    // Nein
```

Achtung

Der Computer zählt ab **0**. Das erste Ding ist `[0]`, das zweite `[1]`.

Wie viele sind es?

.length verrät dir, wie viele Dinge in der Liste sind:

```
console.log(antworten.length) // 3
```

Das brauchen wir gleich, damit der Zufall zur Liste passt.

Übung: deine Liste



ÜBUNG

ca. 10 Minuten

Baue eine Liste mit deinen drei Lieblingstieren und gib das erste aus:

```
const tiere = ["...", "...", "..."]  
console.log(tiere[0])
```

★ **Bonus:** Gib auch aus, wie viele Tiere in der Liste sind (`.length`).

Übung: die richtige Position



ÜBUNG

ca. 8 Minuten

Baue eine Liste mit Wochentagen und gib den **dritten** aus (zählen ab 0!):

```
const tage = ["Mo", "Di", "Mi", "Do", "Fr"]  
console.log(tage[...]) // welcher Index ist der dritte?
```

★ **Bonus:** Gib den letzten Tag aus (Tipp: `tage.length - 1`).



Kapitel 2

Zufall: der Computer würfelt



Der Zufall: Math.random

`Math.random()` gibt eine Zufallszahl zwischen 0 und fast 1:

```
console.log(Math.random()) // z.B. 0.42, dann 0.91, dann  
0.05 ...
```

Jedes Mal eine andere. Nur: so eine Kommazahl können wir schlecht gebrauchen
– wir wollen 0, 1 oder 2.

Vom Zufall zur Auswahl-Zahl

In zwei Schritten wird daraus eine ganze Zahl:

```
Math.random() * 3      // 0 bis fast 3, z.B. 2.4  
Math.floor(2.4)        // abrunden -> 2
```

`Math.floor` rundet immer **ab**. Zusammen ergibt das 0, 1 oder 2:

```
const zufall = Math.floor(Math.random() * 3)
```

Übung: würfeln in der Konsole



ÜBUNG

ca. 8 Minuten

Lass den Computer eine Zufallszahl 0, 1 oder 2 ausgeben. Führe es mehrmals aus!

```
console.log(Math.floor(Math.random() * 3))
```

★ **Bonus:** Mach einen echten Würfel: eine Zahl von 1 bis 6 (Tipp: am Ende + 1).

Ein zufälliges Ding aus der Liste

Jetzt Liste und Zufall zusammen. Statt der 3 nimmst du `.length`:

```
const antworten = ["Ja", "Nein", "Vielleicht"]  
  
const i = Math.floor(Math.random() * antworten.length)  
console.log(antworten[i])    // jedes Mal anders
```

Führ es mehrmals aus. Der Computer würfelt eine Antwort.



Projekt

Die Wahrsager-Kugel



Jetzt öffnen: [3a-wahrsager](#)

in VS Code – oder auf der Kursseite



So ist deine Vorlage aufgebaut

```
<h2 id="antwort">Frag die Kugel...</h2>  
<button onclick="frage()">Frag die Kugel</button>  
  
<script src="script.js"></script>
```

Eine Anzeige namens **antwort** und ein Knopf, der `frage()` ruft. Die Funktion schreibst du in `script.js`.

Die Kugel antwortet

```
const antworten = ["...", "...", "..."] // deine Antworten

function frage() {
  const i = Math.floor(Math.random() * antworten.length)
  document.getElementById("antwort").innerHTML = antworten[i]
}
```

Kennst du alles: Liste, Zufall, Knopf, innerHTML. Neu ist nur, dass sie zusammenspielen.

Fehler-Detektive



FINDE DEN FEHLER

Die Kugel zeigt manchmal „undefined“. Warum?

```
const antworten = ["Ja", "Nein"]  
  
const i = Math.floor(Math.random() * 5)  
console.log(antworten[i])
```

Tipp: Passt die 5 zur Länge der Liste?

Übung: deine eigene Kugel



PROJEKT

ca. 15 Minuten

- Fülle die Funktion `frage( )` aus
- Schreibe deine eigenen Antworten in die Liste

★ **Bonus:** Baue eine Themen-Kugel: eine Gute-Laune-Kugel oder eine Was-esse-ich-heute-Kugel mit mindestens sechs Antworten.

Zeig deine Kugel!

Ja, ganz sicher!

Frag die Kugel

Frag die Kugel ein paar Mal und zeig sie den anderen.



Kapitel 3

Entscheidungen: der Computer wählt

Wenn ... dann

Ein `if` prüft, ob etwas stimmt. Wenn ja, läuft der Code dazwischen:

```
const alter = 12

if (alter >= 10) {
  console.log("Du bist schon gross!")
}
```

In den runden Klammern steht die **Bedingung**. Stimmt sie nicht, wird alles übersprungen.

Womit du vergleichst

===	=	ist genau gleich
>	=	größer
<	=	kleiner
>=	=	größer oder gleich
<=	=	kleiner oder gleich

Zum Vergleichen === mit drei Zeichen, nicht = mit einem.

🔴 Wahr oder falsch? (boolean)

So ein Vergleich ist immer entweder **wahr** oder **falsch**. Das nennt man einen **boolean**:

```
console.log(10 > 3)      // true  (wahr)
console.log(2 === 5)     // false (falsch)
```

Nur wenn die Bedingung `true` ist, läuft der Code im `if`. Du kannst `true` oder `false` auch in einer Variable merken:

```
let gewonnen = false
```

Übung: wahr oder falsch?



ÜBUNG

ca. 8 Minuten

Rate erst, dann teste in der Konsole – kommt `true` oder `false`?

```
console.log(7 > 2)
console.log(3 === 5)
console.log(10 <= 10)
```

★ **Bonus:** Merk dir ein Ergebnis in einer Variable: `let gross = 100 > 50.`

if und else

Meistens willst du zwei Wege: einen, wenn es stimmt, und einen, wenn nicht.

```
if (alter >= 10) {  
  console.log("Du bist schon gross!")  
} else {  
  console.log("Du wirst noch gross!")  
}
```

Genau ein Weg läuft, nie beide.

Mehrere Wege: else if

```
if (punkte >= 90) {  
  console.log("Super!")  
} else if (punkte >= 50) {  
  console.log("Gut gemacht.")  
} else {  
  console.log("Weiter üben!")  
}
```

Der Computer nimmt den **ersten** Weg, der stimmt.

Übung: triff eine Entscheidung



ÜBUNG

ca. 10 Minuten

Schreibe ein `if/else`: wenn `alter >= 12`, gib `"Du darfst!"` aus, sonst `"Noch nicht."`

```
let alter = 12
if (...) {
  // ...
} else {
  // ...
}
```

★ **Bonus:** Mach drei Fälle mit `else if` (Kind, Teenager, Erwachsener).

Und & Oder: zwei Bedingungen

Mit `&&` (und) und `||` (oder) prüfst du zwei Dinge auf einmal:

```
if (alter >= 6 && alter <= 12) {    // beide müssen stimmen
  console.log("Genau richtig!")
}
if (tag === "Sa" || tag === "So") {  // eines reicht
  console.log("Wochenende!")
}
```

`&&` = **beide** zusammen (Hausaufgaben *und* Zimmer). `||` = **eines** reicht (Sa *oder* So).

Übung: und & oder



ÜBUNG

ca. 8 Minuten

Prüfe mit `&&`: wenn `leben > 0` und `punkte >= 100`, gib "Level geschafft!" aus.

```
let leben = 3
let punkte = 120

if (... && ...) {
  console.log("Level geschafft!")
}
```

★ **Bonus:** Mit `||`: gewinne, wenn `gegner === 0` oder `zeit === 0` ist.

Fehler-Detektive



FINDE DEN FEHLER

Diese Abfrage stimmt immer, egal welche Zahl. Warum?

```
if (zahl = 10) {  
  console.log("Die Zahl ist zehn")  
}
```

Tipp: Ein = befüllt, drei === vergleichen.



Projekt

Das Glücksrad



Jetzt öffnen: [3b-gluecksrاد](#)

in VS Code – oder auf der Kursseite



So ist deine Vorlage aufgebaut

```
<h2 id="rad">- - -</h2>
<p id="text">Dreh am Rad!</p>
<button onclick="dreh()">Drehen</button>
```

Ein Feld **rad** für die Symbole, eine Nachricht und ein Knopf, der `dreh()` ruft.

★ Drei Zufalls-Symbole

Wir würfeln dreimal aus einer Liste von Symbolen:

```
const symbole = ["...", "...", "..."] // deine Symbole

function dreh() {
  const a = symbole[Math.floor(Math.random() * 3)]
  const b = ... // genau wie a
  const c = ... // genau wie a
  document.getElementById("rad").innerHTML = a + " " + b + " "
    + c
}
```

Drei Gleiche = Jackpot

Am Ende von `dreh()` prüfen wir, ob alle drei gleich sind:

```
if (a === b && b === c) {  
    document.getElementById("text").innerHTML = "JACKPOT!"  
} else {  
    document.getElementById("text").innerHTML = "Nochmal!"  
}
```

`&&` heißt „und“: nur wenn *alle* drei gleich sind, gibt es den Jackpot.

Übung: dein Glücksrad



PROJEKT

ca. 12 Minuten

- Bring das Glücksrad zum Laufen
- Schreibe deine eigenen Symbole in die Liste (im Browser gehen auch Emojis)

★ **Bonus:** Zähle mit einem `let`-Zähler, wie oft du den Jackpot geknackt hast.

★ Zeig dein Glücksrad!

Stern Stern Stern
JACKPOT!

Drehen

Kurz erinnert: Parameter

Aus Modul 2 kennst du **Parameter** schon: eine Funktion kann in den Klammern eine Info mitbekommen.

Ein Knopf kann der Funktion sagen: `raten("hoeher")`

Das brauchst du gleich in mehreren Spielen wieder.



Projekt

Höher oder Tiefer



Jetzt öffnen: [3c-hoeher-tiefer](#)

in VS Code – oder auf der Kursseite

Wie es geht

- du siehst eine Karte (eine Zahl von 1 bis 13)
- du tippst, ob die nächste **höher** oder **tiefer** ist
- richtig getippt gibt einen Punkt, daneben geht es zurück auf 0

Nur Zufall und `if`, alles, was du in diesem Modul gelernt hast.



So ist deine Vorlage aufgebaut

```
<h2 id="karte"></h2>
<p id="text">Höher oder tiefer?</p>
<p id="punkte">Punkte: 0</p>
<button onclick="raten('hoeher')">Höher</button>
<button onclick="raten('tiefer')">Tiefer</button>
```

Ein Feld für die **karte**, eine Punkte-Anzeige und zwei Knöpfe, die `raten(...)` rufen.

Die neue Karte

```
let aktuelle = Math.floor(Math.random() * 13) + 1
let punkte = 0

function raten(wahl) {
  const naechste = Math.floor(Math.random() * 13) + 1
  let richtig = false
  if (wahl === "hoeher" && naechste > aktuelle) richtig = true
  // ... jetzt das Gleiche für "tiefer" (naechste < aktuelle)
}
```

richtig merkt sich, ob du recht hattest.

Punkte zählen

Ans Ende von raten kommt: Punkte hoch oder zurück auf 0, dann neue Karte.

```
if (richtig) {  
    punkte = punkte + 1  
} else {  
    punkte = 0  
}  
aktuelle = naechste  
document.getElementById("karte").innerHTML = aktuelle  
document.getElementById("punkte").innerHTML = "Punkte: " +  
    punkte
```

Übung: dein Karten-Spiel



PROJEKT

ca. 12 Minuten

- Bring Höher oder Tiefer zum Laufen
- Wie viele Karten schaffst du am Stück?

★ **Bonus:** Zeig eine Nachricht wie „Richtig!“ oder „Daneben!“ im Feld text.

Zeig dein Karten-Spiel!

9

Punkte: 4

Höher

Tiefer



Projekt

Der Taschenrechner



Jetzt öffnen: [3d-taschenrechner](#)

in VS Code – oder auf der Kursseite



So ist deine Vorlage aufgebaut

```
<h2 id="display">0</h2>
<button onclick="tippe('1')">1</button>
<button onclick="tippe('2')">2</button>    <!-- ... 3 bis 9 und 0 -->
<button onclick="waehle('+')">+</button>
<button onclick="waehle('-')">-</button>
<button onclick="rechne()">=</button>
```

Zahl-Knöpfe rufen `tippe(...)`, Rechen-Knöpfe `waehle(...)`, das Gleich `rechne()`.

Ziffern tippen

Jeder Zahl-Knopf hängt seine Ziffer an die Anzeige an:

```
let anzeige = ""

function tippe(ziffer) {
  anzeige = anzeige + ziffer
  document.getElementById("display").innerHTML = anzeige
}
```

Der Parameter `ziffer` sagt der Funktion, welche Zahl gedrückt wurde.

Rechenart merken

Ein Rechen-Knopf merkt sich die erste Zahl und die Rechenart, dann macht er die Anzeige frei:

```
let ersteZahl = 0
let rechenart = ""

function waehle(art) {
  ersteZahl = Number(anzeige)
  rechenart = art
  anzeige = ""
}
```

Ausrechnen

Das Gleich rechnet, je nach Rechenart. Dein `if/else`, nur mit Rechnen:

```
function rechne() {  
  const zweite = Number(anzeige)  
  let ergebnis = 0  
  if (rechenart === "+") {  
    ergebnis = ersteZahl + zweite  
  } else if (rechenart === "-") {  
    ergebnis = ersteZahl - zweite  
  }  
  document.getElementById("display").innerHTML = ergebnis  
}
```

Übung: dein Taschenrechner



PROJEKT

ca. 15 Minuten

- Bring Tippen, Wählen und Rechnen zum Laufen
- Teste ein paar Rechnungen

★ **Bonus:** Baue auch Mal (*) und Geteilt (/) ein, je ein weiteres `else if`.



Projekt

Schere, Stein, Papier



Jetzt öffnen: [3e-schere-stein-papier](#)

in VS Code – oder auf der Kursseite



So ist deine Vorlage aufgebaut

```
<h2 id="ergebnis">Wähle!</h2>
```

```
<button onclick="spiele('Schere')">Schere</button>
```

```
<button onclick="spiele('Stein')">Stein</button>
```

```
<button onclick="spiele('Papier')">Papier</button>
```

Jeder Knopf ruft `spiele(...)` auf und sagt mit dem Parameter, was du gewählt hast.

Der Computer wählt zufällig

```
function spiele(spieler) {  
  const optionen = ["Schere", "Stein", "Papier"]  
  const computer = optionen[Math.floor(Math.random() * 3)]  
  
  // ... jetzt vergleichen wir spieler und computer  
}
```

Der spieler kommt vom Knopf, der computer wird gewürfelt.

Wer gewinnt?

```
if (spieler === computer) {  
  ergebnis = "Unentschieden!"  
} else if (  
  (spieler === "Schere" && computer === "Papier") ||  
  // ... und die zwei anderen Gewinn-Kombis findest du selbst  
) {  
  ergebnis = "Du gewinnst!"  
}  
// sonst gewinnt der Computer
```

Bei einer der drei richtigen Kombinationen gewinnst du, sonst der Computer.

Übung: mach es zu deinem Spiel



PROJEKT

ca. 15 Minuten

- Bring das Spiel zum Laufen und spiel ein paar Runden
- Ändere die Ergebnis-Texte, wie du magst

★ **Bonus:** Zähle mit einem `let`-Zähler, wie oft du gewonnen hast, und zeig es an.

Zeig dein Spiel!

Du gewinnst!

Schere

Stein

Papier

Spiel ein paar Runden gegen den Computer und fordere die anderen heraus.

✓ Das kannst du jetzt

- **Listen** bauen und Dinge über die Position holen
- den Computer mit `Math.random` **würfeln** lassen
- mit `if, else if` und `else` **entscheiden**
- mehrere eigene **Spiele** und Werkzeuge bauen

Jetzt weiß der Computer vorher nicht, was passiert. Das macht Spiele spannend!

☰ Neue Befehle auf einen Blick

<code>["a", "b", "c"]</code>	=	eine Liste bauen
<code>liste[o]</code>	=	das erste Ding (ab 0!)
<code>liste.length</code>	=	wie viele Dinge
<code>Math.random()</code>	=	würfeln
<code>if / else if / else</code>	=	entscheiden
<code>===</code>	=	ist genau gleich



Als Nächstes: Modul 4

Dein Computer kann würfeln und entscheiden. Als Nächstes lernt er, Dinge **viele Male** zu tun:

Schleifen, ohne alles abzutippen.

Damit werden deine Programme richtig groß.